

Universidad Centroamericana José Simeón Cañas

Facultad de Ingeniería y Arquitectura - Ingeniería informática



Universidad
Centroamericana
José Simeón Cañas

Ing. James Edward Humberstone Morales

Proyecto de Cátedra – Administración de Bases de Datos con SQL Server: “Sistema de reserva de Gimnasio”

Integrantes: Fidel Ignacio Peralta Rivas 00196321
Gabriel Alexander Lira Andrade 00098122
María Renée Salmerón Avendaño 00142524
Oscar Benjamín Díaz Escalante 00301423
Pamela Alexandra Gómez Tenorio 00139622

Fecha: 26 de noviembre de 2025

Descripción del sistema elegido y modelo de datos (ER).

En nuestro caso hemos escogido el sistema de reserva de gimnasio. Nuestro sistema permite a una empresa gestionar a sus socios, entrenadores, clases y pagos asociados a la membresía del plan escogido.

- **Gestionar a sus socios:** (al momento de referirnos a un socio se refiere a cualquier persona que haga uso y consumo de productos, instalaciones y de beneficios asociado a los planes del gimnasio) el registro de un nuevo socio al gimnasio, el seguimiento y constante actualización de su información personal (de ser necesario), el consumo del plan que el socio ha escogido y su pagos asociados a su consumo.
- **Gestionar entrenadores:** registro de entrenadores al momento de ser contratados, sus horarios disponibles y las clases asociadas a cada entrenador
- **Gestionar clases:** creación de nuevas clases con toda la información que ello conlleva (horarios impartidos, cupos, tipos de clase), relación de la clase con su respectivo entrenador
- **Gestionar reservas de los socios:** los socios podrán reservar un cupo a clases que deseen asistir dependiendo de su disponibilidad y estado de pago asociado al plan que han decidido consumir
- **Gestionar pagos de socios y otros:** se podrá llevar un registro del estado de pago de cada socio, el control de membresías activas asociadas a sus planes, generar reportes de calidad y auditar dichos pagos de ser necesarios, control de pagos y salarios asociados también a un entrenador con su seguro y demás.

Haciendo uso de SQL Server el sistema permite hacer consultas rápidas seguras, siendo estas apoyadas por distintos tipos de índices y funciones ventana así como otras estrategias de optimización para así garantizar una mejor optimización de consultas u operaciones frecuentes que se necesiten por ejemplo: la búsqueda de clases disponibles o el listado de socios activos. También, tiene una integración con Power BI para poder visualizar estadísticas clave que conciernen a la organización como el número de reservas hechas al mes de una clase en específico, la asistencia promedio de dicha clase o los ingresos mensuales que se reciben de cada plan.

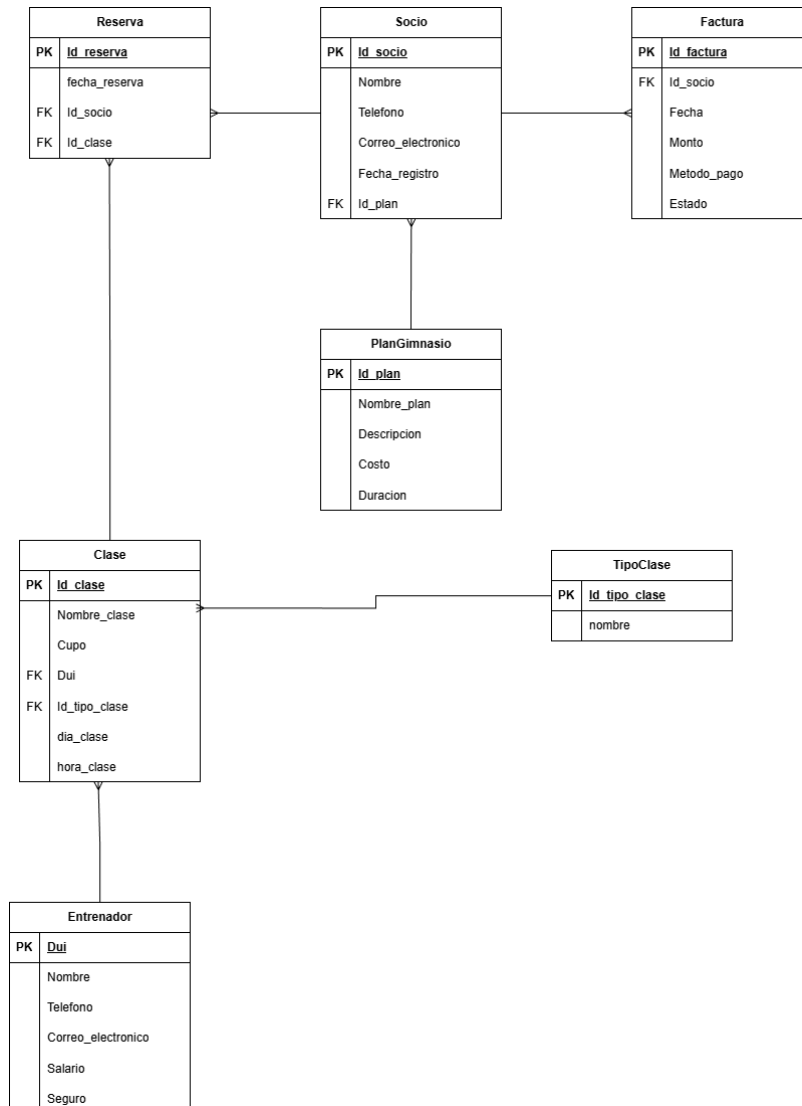
El modelo ER se basa en Entidades principales y sus Relaciones (ER) en nuestro caso tenemos:

- **Socio:** se almacena información personal y su consumo asociado a su plan y se relaciona con la factura que paga al pagar su plan, el plan que decide consumir la reserva que hace si desea asistir a una clase en específico.
- **Entrenador:** así como el socio se almacena su información personal y el horario al momento de impartir alguna clase este se relaciona con la clase en sí que el entrenador imparte.
- **Clase:** se guarda información crucial a la clase como el horario, el cupo de la clase y el documento de Identidad asociada a las personas que asisten a la clase como otros datos, todo esto se relaciona con el entrenador que imparte la clase, el tipo de clase que se provee y la reserva que hace un socio para asistir a la clase.
- **Tipo de clase:** se almacena el tipo de clase que se provee así como su nombre, esto solo se relaciona con la clase en sí a la que pertenece.
- **Reserva:** se almacena información clave relacionada a la reserva como la fecha, datos de la persona que hace la reserva, etc y todo esto se relaciona con la clase a la que se hace la reserva y el socio que hace dicha reserva.
- **Plan del gimnasio:** se guardan los datos asociados a cada plan disponible con sus costos y datos importantes, solo se relaciona el socio que consume y paga dicho plan.

- **Factura:** se guardan todos los comprobantes de los pagos realizados hacia la empresa a cambio del consumo de productos y servicios que se proveen, así como el estado de pago de los socios, esto solo se relaciona con el socio que hace dicho pago.

Algunas de las relaciones más importantes son: la relación del socio con su reservas y pagos ya que el socio puede hacer muchas reservas a muchas clases y a su vez hacer múltiples pagos a lo largo del tiempo, la clase que se imparte puede tener a muchos socios que asisten a ella, un entrenador puede dar solo una o múltiples clases y un plan puede ser consumido por múltiples socios, entre otros.

Aquí se presenta un esquema de la base de datos:



Políticas de seguridad implementadas.

La base de datos ha sido organizada en esquemas, con el objetivo de modularizar los objetos de la misma en una división funcional y coherente que permita delegar seguridad a nivel de cada esquema, evitar conflictos de nombres, mejorar la mantenibilidad y claridad del modelo y permitir la migración o respaldo selectivo.

Esquema	Objetos
gestion_clases	Clase, Entrenador, TipoClase, Reserva
gestion_socios	Socio, PlanGimnasio
contabilidad	Factura

Para facilitar la gestión de privilegios se han creado los siguientes roles

Rol	Esquema	Objeto	SELECT	INSERT	UPDATE	DELETE	CREATE	ALTER	DROP	BACKUP	EXECUTE
gerente_operaciones	gestion_clases	clase	☒	☒	☒	☒	☐	☐	☐	☐	☐
		entrenador	☒	☒	☒	☒	☐	☐	☐	☐	☐
		tipoClase	☒	☒	☒	☒	☐	☐	☐	☐	☐
		reserva	☒	☒	☒	☒	☐	☐	☐	☐	☐
	gestion_socios	socio	☒	☒	☒	☐	☐	☐	☐	☐	☐
		planGimnasio	☒	☐	☒	☐	☐	☐	☐	☐	☐
	contabilidad	factura	☒	☐	☐	☐	☐	☐	☐	☐	☐
analista_reportes	gestion_clases	clase	☒	☐	☐	☐	☐	☐	☐	☐	☐
		entrenador	☒	☐	☐	☐	☐	☐	☐	☐	☐
		tipoClase	☒	☐	☐	☐	☐	☐	☐	☐	☐
		reserva	☒	☐	☐	☐	☐	☐	☐	☐	☐
	gestion_socios	socio	☒	☐	☐	☐	☐	☐	☐	☐	☐
		planGimnasio	☒	☐	☐	☐	☐	☐	☐	☐	☐
	contabilidad	factura	☒	☐	☐	☐	☐	☐	☐	☐	☐
rol_administrador	gestion_clases	clase	☒	☒	☒	☒	☒	☒	☒	☒	☒
		entrenador	☒	☒	☒	☒	☒	☒	☒	☒	☒
		tipoClase	☒	☒	☒	☒	☒	☒	☒	☒	☒
		reserva	☒	☒	☒	☒	☒	☒	☒	☒	☒
	gestion_socios	socio	☒	☒	☒	☒	☒	☒	☒	☒	☒
		planGimnasio	☒	☒	☒	☒	☒	☒	☒	☒	☒
	contabilidad	factura	☒	☒	☒	☒	☒	☒	☒	☒	☒
rol_backup	gestion_clases	clase	☐	☐	☐	☐	☐	☐	☐	☒	☐
		entrenador	☐	☐	☐	☐	☐	☐	☐	☒	☐
		tipoClase	☐	☐	☐	☐	☐	☐	☐	☒	☐
		reserva	☐	☐	☐	☐	☐	☐	☐	☒	☐
	gestion_socios	socio	☐	☐	☐	☐	☐	☐	☐	☒	☐
		planGimnasio	☐	☐	☐	☐	☐	☐	☐	☒	☐
	contabilidad	factura	☐	☐	☐	☐	☐	☐	☐	☒	☐

Política de Seguridad para Contraseñas:

- La complejidad requerida será de mínimo 12 caracteres con mayúsculas, minúsculas, números y símbolos.
- Con rotación periódica obligatoria.
- Mediante CHECK_POLICY = ON y CHECK_EXPIRATION = ON.

Con el fin de garantizar la integridad y confidencialidad de los datos, a través de una autenticación robusta.

- rol_administrador: Administrador de base de datos con control total, con la capacidad de CREATE, ALTER, DROP, DELETE en todos los esquemas, GRANT, REVOKE, realizar gestión de usuarios y roles y un acceso completo a nivel servidor.
- rol_backup: permiso para backup únicamente para realizar respaldos y restauraciones
- rol_gerente_operaciones: supervisión operativa completa, con permiso de SELECT en todos los esquemas, INSERT, UPDATE en gestion_clases.
- rol_analista_reportes: con permiso SELECT en todos los esquemas más vistas de administración.

Rol	Usuario	Contraseña de usuario
rol_administrador	admin_db	c0n54\$i_1L23K85k
	estudiantes_db	M#ripPl?azsJNyUt
rol_backup	backup_user	67gt56h_19?jTkYa
rol_gerente_operaciones	gerente_operaciones	Lp?!lakA21akj%rB!
rol_analista_reportes	analista_reportes	rRryT278\$aLmzI_q
	usuario_profesor	sS%10?lOopqnmM

SQLAudit Factura

Con el objetivo de garantizar la trazabilidad y control de las operaciones realizadas sobre la tabla Factura, se implementó un mecanismo de auditoría mediante la creación de un Server Audit y una Database Audit Specification. Esta auditoría permite registrar automáticamente todas las acciones de inserción, actualización y eliminación efectuadas sobre la tabla, junto con la información relevante como el usuario que ejecutó la operación, y la fecha y hora del evento. Los registros generados se encuentran ubicados en una ruta específica del servidor, lo que facilita la revisión posterior. De esta manera, se fortalece la seguridad e integridad de los datos, permitiendo detectar modificaciones no autorizadas y mantener un historial confiable de los cambios efectuados a los datos de facturación, área fundamental para el funcionamiento y sostenibilidad del negocio.

```
USE master;
GO
```

```
CREATE SERVER AUDIT AuditoriaBasica
TO FILE (
    FILEPATH = 'C:\AuditLogs\Factura\',
    MAXSIZE = 20 MB,
    MAX_FILES = 20
)
WITH (ON_FAILURE = CONTINUE);
GO
```

```
ALTER SERVER AUDIT AuditoriaBasica WITH (STATE = ON);
```

```
USE Gimnasio;
GO
```

```
CREATE DATABASE AUDIT SPECIFICATION AuditoriaBasicaFactura
FOR SERVER AUDIT AuditoriaBasica
ADD (INSERT ON OBJECT::dbo.Factura BY PUBLIC),
ADD (UPDATE ON OBJECT::dbo.Factura BY PUBLIC),
ADD (DELETE ON OBJECT::dbo.Factura BY PUBLIC);
-- ADD (SELECT ON OBJECT::dbo.Factura BY PUBLIC);
```

```
ALTER DATABASE AUDIT SPECIFICATION AuditoriaBasicaFactura WITH (STATE =
ON)
```

```
SELECT * FROM sys.fn_get_audit_file('C:\AuditLogs\Factura\*', DEFAULT,
DEFAULT);
```

```
SELECT
    event_time,
    action_id,
    succeeded,
    server_principal_name,
    database_name,
    schema_name, object_name,
    statement,
    additional_information
```

```
FROM sys.fn_get_audit_file('C:\AuditLogs\Factura\*', DEFAULT, DEFAULT)
ORDER BY event_time DESC;
```

Evidencias de consultas implementadas e índices aplicados

En esta sección se detallan las consultas avanzadas desarrolladas para el análisis de datos del gimnasio, haciendo uso de **Funciones de Ventana (Window Functions)** para realizar cálculos complejos de agregación y ordenamiento sin perder el detalle de las filas. Asimismo, se documenta la estrategia de indexación aplicada para optimizar el costo computacional de cada operación.

Reporte de Tendencia de Facturación y Crecimiento Mensual

Descripción del reporte: Este análisis permite visualizar el flujo de ingresos agrupado mensualmente. Su valor agregado reside en el cálculo del **crecimiento absoluto y porcentual** respecto al mes anterior, lo cual es fundamental para la toma de decisiones financieras. Se utiliza la función ventana `LAG()` para acceder a la fila previa (mes anterior) dentro del mismo conjunto de resultados, evitando subconsultas complejas.

Código SQL Implementado:

```
SELECT
    FORMAT(f.Fecha, 'yyyy-MM') as mes,
    SUM(f.Monto) as facturacion_mes,
    LAG(SUM(f.Monto)) OVER(ORDER BY MIN(f.Fecha)) as
facturacion_mes_anterior,
    SUM(f.Monto) - LAG(SUM(f.Monto)) OVER(ORDER BY MIN(f.Fecha)) as
crecimiento_absoluto,
    ((SUM(f.Monto) - LAG(SUM(f.Monto)) OVER(ORDER BY MIN(f.Fecha)))
    * 100.0 / LAG(SUM(f.Monto)) OVER(ORDER BY MIN(f.Fecha))) as
crecimiento_porcentual
FROM contabilidad.Factura f
WHERE f.Estado = 'Pagada'
GROUP BY FORMAT(f.Fecha, 'yyyy-MM')
ORDER BY mes;
```

Optimización aplicada (Índices):

- **Índice:** `IX_Factura_Estado_Fecha`
- **Definición:**

```
CREATE NONCLUSTERED INDEX IX_Factura_Estado_Fecha ON
contabilidad.Factura (Estado, Fecha);
```

Se creó un índice compuesto. La columna `Estado` se coloca primero para satisfacer inmediatamente el filtro `WHERE`. La columna `Fecha` permite que la función ventana `LAG(... OVER(ORDER BY ...))` encuentre los datos ya ordenados cronológicamente sin realizar una operación de ordenamiento en memoria (*Sort Operator*).

Resultado de la consulta (primeros 10 registros):

	mes	facturacion_mes	facturacion_mes_anterior	crecimiento_absoluto	crecimiento_porcentual
1	2020-11	582975.29	NULL	NULL	NULL
2	2020-12	1095637.19	582975.29	512661.90	87.938873
3	2021-01	1087433.78	1095637.19	-8203.41	-0.748734
4	2021-02	1000587.27	1087433.78	-86846.51	-7.986372
5	2021-03	1116165.39	1000587.27	115578.12	11.551028
6	2021-04	1065548.33	1116165.39	-50617.06	-4.534906
7	2021-05	1172095.96	1065548.33	106547.63	9.999324
8	2021-06	1092859.58	1172095.96	-79236.38	-6.760229
9	2021-07	1103704.17	1092859.58	10844.59	0.992313
10	2021-08	1086836.16	1103704.17	-16868.01	-1.528308

Ranking Estratégico de Entrenadores por Salario

Descripción del reporte: Este reporte jerarquiza a los entrenadores basándose en su compensación económica. Utiliza la función `RANK()` para asignar una posición numérica y `SUM()` `OVER()` para calcular el total de la nómina global dinámicamente. Esto permite visualizar qué porcentaje del presupuesto total representa el salario de cada entrenador individual.

Código SQL Implementado:

```
SELECT
    e.Dui,
    e.Nombre,
    e.Salario,
    RANK() OVER(ORDER BY e.Salario DESC) AS ranking_salario,
    ROUND(e.Salario * 100.0 / SUM(e.Salario) OVER(), 2) AS
porcentaje_del_total_salarial
FROM gestion_clases.Entrenador e
LEFT JOIN gestion_clases.Clase c ON e.Dui = c.Dui
GROUP BY e.Dui, e.Nombre, e.Salario, e.Seguro
ORDER BY e.Salario DESC;
```

Optimización aplicada (Índices):

- **Índice principal:** `IX_Entrenador_Salario`
- **Definición:**

```
CREATE NONCLUSTERED INDEX IX_Entrenador_Salario ON
gestion_clases.Entrenador (Salario DESC) INCLUDE (Nombre);
```

La función ventana requiere ordenar los datos por salario de forma descendente. Al crear el índice con `Salario DESC`, almacenamos los datos físicamente en el orden exacto que la consulta necesita. Esto elimina la carga de procesamiento del CPU para ordenar los registros antes de asignar el ranking. La inclusión del `Nombre` permite obtener los datos descriptivos sin realizar un *Key Lookup* a la tabla principal.

- **Índice de Soporte:** `IX_Clase_Dui`

- **Definición:**

```
CREATE NONCLUSTERED INDEX IX_Clase_Dui ON gestion_clases.Clase
(Dui);
```

Este índice acelera el LEFT JOIN con la tabla Clase. Es importante notar que este es un **índice compartido**, ya que también es vital para optimizar las relaciones en el "Ranking Mensual de Entrenadores" (Consulta 4).

Resultado de la consulta (todos los entrenadores):

	Dui	Nombre	Salario	ranking_salario	porcentaje_del_total_salarial
1	477606022	Jose Pedro	2988.57	1	7.86000000000000000000000000000000
2	678752568	Sara Guerrero	2892.78	2	7.61000000000000000000000000000000
3	311376824	Pili Arrieta	2643.29	3	6.95000000000000000000000000000000
4	542816047	María Ángeles Azorin	2573.24	4	6.77000000000000000000000000000000
5	639174721	Ana Belén Vallejo	2551.64	5	6.71000000000000000000000000000000
6	799354163	Guadalupe Puig	2543.00	6	6.69000000000000000000000000000000
7	498528749	Eufemia Marqués	2346.54	7	6.17000000000000000000000000000000
8	722521836	Jose Ramón Jover	2093.49	8	5.51000000000000000000000000000000
9	132184473	Encarnacion Calzada	1968.76	9	5.18000000000000000000000000000000
10	964398950	Florencia Palmer	1807.56	10	4.76000000000000000000000000000000
11	982952537	Constanza Trujillo	1753.09	11	4.61000000000000000000000000000000
12	343357553	Emilio Quevedo	1737.70	12	4.57000000000000000000000000000000
13	243001616	Paulina Garrido	1722.36	13	4.53000000000000000000000000000000
14	283036477	Ignacia Galván	1632.68	14	4.30000000000000000000000000000000
15	139887618	Jose Miguel Vilaplana	1461.89	15	3.85000000000000000000000000000000
16	711661802	Maxi Agullo	1402.56	16	3.69000000000000000000000000000000
17	224390632	Angelina Ibarra	1207.81	17	3.18000000000000000000000000000000
18	132957606	Marco Girona	920.10	18	2.42000000000000000000000000000000
19	721974441	Alicia Oliveras	919.93	19	2.42000000000000000000000000000000
20	882748004	Rico Gual	841.67	20	2.21000000000000000000000000000000

Ranking de Horarios Preferidos por Tipo de Clase

Descripción del reporte: El objetivo es identificar los "horarios populares" para cada disciplina o tipo de clase (Yoga, Crossfit, etc.). Se utiliza DENSE_RANK() particionado por Tipo de Clase. Esto permite reiniciar el ranking para cada tipo, destacando los horarios con mayor afluencia.

Código SQL Implementado:

```
SELECT
    tc.Nombre AS Tipo_De_Clase,
    c.hora_clase,
    COUNT(r.Id_reserva) AS Total_Reservas,
    DENSE_RANK() OVER (
        PARTITION BY tc.Nombre
        ORDER BY COUNT(r.Id_reserva) DESC
    ) AS Ranking_Horario
FROM gestion_clases.Clase c
JOIN gestion_clases.TipoClase tc ON c.Id_tipo_clase = tc.Id_tipo_clase
LEFT JOIN gestion_clases.Reserva r ON c.Id_clase = r.Id_clase
GROUP BY tc.Nombre, c.hora_clase
ORDER BY tc.Nombre, Ranking_Horario;
```

Optimización aplicada (Índices):

- **Índice Principal:** IX_Clase_IdTipo_Hora
- **Definición:**

```
CREATE NONCLUSTERED INDEX IX_Clase_IdTipo_Hora
ON gestion_clases.Clase (Id_tipo_clase, hora_clase);
```

Dado que la consulta agrupa y particiona la ventana por el tipo de clase, este índice es crítico. Organiza la tabla Clase primero por Id_tipo_clase y luego por hora_clase. Esto facilita la unión (JOIN) con la tabla TipoClase y permite al motor de base de datos realizar las agregaciones de forma secuencial y eficiente (*Stream Aggregate*), optimizando el cálculo del ranking dentro de cada partición.

- **Índice de Soporte:** IX_Reserva_IdClase_Fecha
- **Definición:**

```
CREATE NONCLUSTERED INDEX IX_Reserva_IdClase_Fecha
ON gestion_clases.Reserva (Id_clase, fecha_reserva);
```

Optimiza el LEFT JOIN hacía la tabla de reservas. Este es un **índice compartido** con la siguiente consulta.

Resultado de la consulta (primeros 20 registros):

	Tipo_De_Clase	hora_clase	Total_Reservas	Ranking_Horario
1	Aeróbicos	17:00:00	614	1
2	Aeróbicos	09:00:00	564	2
3	Aeróbicos	05:00:00	533	3
4	Aeróbicos	23:00:00	525	4
5	Aeróbicos	11:00:00	523	5
6	Aeróbicos	14:00:00	520	6
7	Aeróbicos	06:00:00	492	7
8	Aeróbicos	08:00:00	488	8
9	Aeróbicos	13:00:00	480	9
10	Aeróbicos	10:00:00	479	10
11	Aeróbicos	16:00:00	472	11
12	Aeróbicos	07:00:00	467	12
13	Aeróbicos	19:00:00	462	13
14	Aeróbicos	22:00:00	450	14
15	Aeróbicos	15:00:00	445	15
16	Aeróbicos	12:00:00	439	16
17	Aeróbicos	18:00:00	435	17
18	Aeróbicos	21:00:00	424	18
19	Aeróbicos	20:00:00	381	19
20	Boxing	06:00:00	470	1

Evolución Mensual del Rendimiento de Entrenadores

Descripción del reporte: Muestra una tabla de clasificación histórica que evalúa el desempeño de los entrenadores mes a mes según la cantidad de clases impartidas. Mediante DENSE_RANK ()

particionado por Año y Mes, se genera una comparación justa donde el rendimiento se evalúa dentro de periodos temporales específicos, permitiendo ver la evolución de la posición de un entrenador a lo largo del tiempo.

Código SQL Implementado:

```
SELECT
    YEAR(r.fecha_reserva) AS Anio,
    DATENAME(MONTH, r.fecha_reserva) AS Mes,
    e.Nombre AS Nombre_Entrenador,
    COUNT(DISTINCT r.fecha_reserva) AS Cantidad_Clases_Impartidas,

    DENSE_RANK() OVER (
        PARTITION BY YEAR(r.fecha_reserva), MONTH(r.fecha_reserva)
        ORDER BY COUNT(DISTINCT r.fecha_reserva) DESC
    ) AS Ranking_Mensual
FROM gestion_clases.Entrenador e
JOIN gestion_clases.Clase c ON e.Dui = c.Dui
JOIN gestion_clases.Reserva r ON c.Id_clase = r.Id_clase
GROUP BY
    YEAR(r.fecha_reserva),
    MONTH(r.fecha_reserva),
    DATENAME(MONTH, r.fecha_reserva),
    e.Nombre
ORDER BY
    Anio DESC,
    MONTH(r.fecha_reserva) DESC,
    Ranking_Mensual ASC;
```

Optimización aplicada (Índices):

- **Índice Principal:** IX_Reserva_IdClase_Fecha (Compartido)

Es fundamental para conectar las clases con sus reservas. Además, al incluir `fecha_reserva` en el índice, se acelera significativamente el cálculo de fechas únicas (`DISTINCT`), el agrupamiento y la partición por fecha (`YEAR`, `MONTH`) requerida por la función ventana, ya que el motor puede acceder rápidamente a los datos temporales necesarios para el ranking.

- **Índice de Soporte:** IX_Clase_Dui (Compartido)

Optimiza la relación entre Entrenador y Clase, evitando escaneos completos de tabla al buscar las clases asignadas a un instructor.

Resultado de la consulta (primeros 20 registros):

	Anio	Mes	Nombre_Entrenador	Cantidad_Clases_Impartidas	Ranking_Mensual
1	2025	December	Paulina Garrido	136	1
2	2025	December	María Ángeles Azorin	131	2
3	2025	December	Jose Ramón Jover	131	2
4	2025	December	Constanza Trujillo	130	3
5	2025	December	Eufemia Marqués	130	3
6	2025	December	Angelina Ibarra	129	4
7	2025	December	Ignacia Galván	126	5
8	2025	December	Guadalupe Puig	125	6
9	2025	December	Encarnacion Calzada	123	7
10	2025	December	Rico Gual	123	7
11	2025	December	Ana Belén Vallejo	121	8
12	2025	December	Pili Arrieta	119	9
13	2025	December	Marco Girona	118	10
14	2025	December	Maxi Agullo	118	10
15	2025	December	Jose Miguel Vilapla...	112	11
16	2025	December	Jose Pedro	111	12
17	2025	December	Emilio Quevedo	111	12
18	2025	December	Florencia Palmer	108	13
19	2025	December	Alicia Oliveras	106	14
20	2025	December	Sara Guerrero	104	15

Diagnóstico de Salud y Fragmentación de Índices

Para garantizar que la estrategia de optimización se mantenga efectiva a lo largo del tiempo, se ha implementado una consulta de diagnóstico técnico. Esta herramienta permite monitorear el estado físico de los índices y detectar problemas de rendimiento causados por el uso continuo de la base de datos.

Consulta de Diagnóstico Implementada:

```
SELECT
    t.name AS Tabla,
    i.name AS Indice,
    i.type_desc AS TipoIndice,
    i.is_primary_key AS EsClavePrimaria,
    i.is_unique AS EsUnico,
    c.name AS Columna,
    ips.avg_fragmentation_in_percent AS Fragmentacion
FROM sys.indexes i
INNER JOIN sys.tables t ON i.object_id = t.object_id
INNER JOIN sys.index_columns ic ON i.object_id = ic.object_id AND
i.index_id = ic.index_id
INNER JOIN sys.columns c ON ic.object_id = c.object_id AND ic.column_id
= c.column_id
OUTER APPLY sys.dm_db_index_physical_stats(DB_ID(), i.object_id,
i.index_id, NULL, 'LIMITED') ips
WHERE t.is_ms_shipped = 0
ORDER BY t.name, i.name, ic.key_ordinal;
```

El propósito fundamental de esta consulta es medir el nivel de **fragmentación** de los índices.

A medida que se insertan, modifican o eliminan registros en el gimnasio (operaciones DML), el orden lógico de los datos deja de coincidir con su orden físico en el disco, generando "huecos" y desorden en las páginas de datos. Este fenómeno se conoce como fragmentación.

Estrategia de dimensionamiento, respaldo y recuperación.

Estrategia de backup híbrida para alta criticidad.

Debido a que el gimnasio opera toda la semana en un horario de 5 de la mañana a 11 de la noche se implementó el horario que se muestra a en la imagen para realizar un plan de backup híbrido que combina los tipos de respaldo completos, incrementales y diferenciales para optimizar el equilibrio entre el espacio de almacenamiento, el tiempo de ejecución y los objetivos de recuperación.

Considerando que las tablas superan los 20,000 registros cada una, esta estrategia permite capturar cambios frecuentes mediante incrementales (4 veces al día) que son rápidos y livianos, establecer puntos de recuperación consolidados con diferenciales (3:00 p.m. diario) que facilitan restauraciones más ágiles, y generar bases sólidas semanales con completos (domingos y jueves a las 2:00 a.m.) que aseguran puntos de recuperación independientes y confiables.

BACKUPS	DOMINGO	LUNES	MARTES	MIÉRCOLES	JUEVES	VIERNES	SÁBADO
1:00 a. m.	Incremental	Incremental	Incremental	Incremental	Incremental	Incremental	Incremental
2:00 a. m.	Completo				Completo		
7:00 a. m.	Incremental	Incremental	Incremental	Incremental	Incremental	Incremental	Incremental
1:00 p. m.	Incremental	Incremental	Incremental	Incremental	Incremental	Incremental	Incremental
3:00 p. m.	Diferencial	Diferencial	Diferencial	Diferencial	Diferencial	Diferencial	Diferencial
7:00 p. m.	Incremental	Incremental	Incremental	Incremental	Incremental	Incremental	Incremental

Beneficios en RTO y RPO ante fallas críticas:

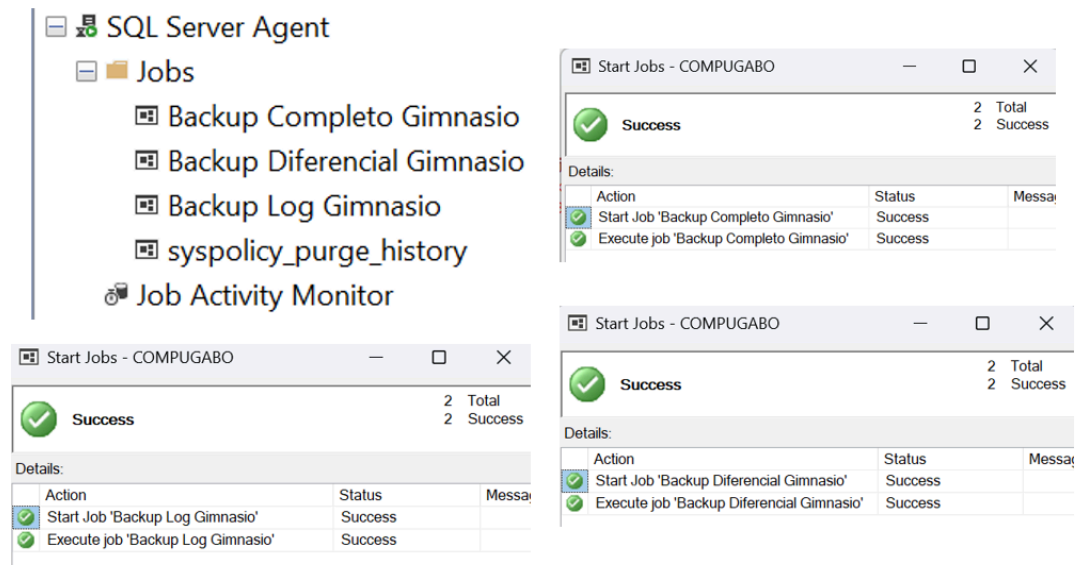
- RPO (Recovery Point Objective): Con backups cada 6 horas, la pérdida máxima de datos sería de 6 horas, protegiendo la integridad de los 20,000 registros por tabla.
- RTO (Recovery Time Objective): La combinación estratégica permite múltiples opciones de recuperación. Para una restauración completa del sistema, se puede usar el último backup completo más el último diferencial e incremental, reduciendo significativamente el tiempo de recuperación comparado con reconstruir desde una larga cadena de incrementales.

Proceso de recuperación completa:

1. Identificar el último backup completo viable (domingo o jueves)
2. Aplicar el último backup diferencial disponible basado en ese completo
3. Aplicar los backups incrementales secuencialmente desde el último diferencial hasta el punto de falla
4. Verificar la integridad de los registros en todas las tablas recuperadas
5. Poner el sistema en producción una vez validada la consistencia de los datos

Esta aproximación híbrida garantiza que, ante una falla catastrófica, la organización pueda recuperarse rápidamente con mínima pérdida de información, manteniendo la integridad de los registros críticos del gimnasio.

Para realizar todo el plan de backup se crearon 3 jobs los cuales se adjuntan como evidencia de que los jobs existentes se ejecutaron exitosamente.



Dimensionamiento

Fórmulas:

Tregistro= $\sum$ (tamaño de cada campo)+Overhead de fila (7 bytes)

Ttabla= Tregistro $\times$ Nregistros

Tabla Clase:

Columna	Tipo de dato	Tamaño en bytes	Cálculo
id_clase	int	4	1
Nombre_clase	nvarchar(100)	200	2 bytes * 100
Cupo	int	4	4
Dui	varchar(10)	10	1 byte * 10
id_tipo_clase	varchar(200)	200	1 byte * 10
dia_clase	tinyint	1	1
hora_clase	time	5	5
Overhead	N/A	7	fila variable

Tregistro=4+200+4+10+200+1+5+7 = 431 bytes

Ttabla=431 $\times$ 35,000=15,085,000 bytes

Convirtiendo a Megabytes: 15,085,000 bytes $\div$ 1024=14,727.54 KB

14,727.54 KB $\div$ 1024=14.38 MB

El tamaño estimado es de 14.38 MB para la tabla Clase

Diccionario de datos:

Columna	Tipo	Restricción	Descripción
Id_clase	int	Primary Key	Llave principal de la clase
Nombre_clase	nvarchar(100)		Nombre de la clase
Cupo	int	Check (Cupo > 0)	Cupos de la clase que deben ser mayor a 0
Dui	varchar(10)		Identificador de una persona que asiste a una clase
Id_tipo_clase	varchar(200)	Foreign Key (Id_tipo_clase)	Llave foránea a tipoclase
dia_clase	tinyint	Check (dia_clase ≥ 1 AND dia ≤ 7)	Día de la clase que debe ser de lunes a domingo
hora_clase	time	hora_clase con minutos=0 y	Hora de la clase que serán cerradas

Tabla Entrenador:

Columna	Tipo de dato	Tamaño en bytes	Cálculo
Dui	varchar(10)	10	1 byte * 10
Nombre	nvarchar(100)	200	2 bytes * 100
Telefono	varchar(50)	50	1 byte * 50
Correo_electronico	varchar(150)	150	decimal: 19 bytes se van a usar 5
Salario	decimal(5,2)	5	1 byte *
Seguro	varchar(100)	100	1 byte * 100
Overhead	N/A	7	fila variable

Tregistro=10+200+50+150+5+100+7 = 522 bytes

Ttabla=522×20=10,440 bytes

Convirtiendo a Megabytes: 10,440 bytes ÷ 1024 = 10.19 KB

10.19 KB ÷ 1024 = 0.009 MB

Diccionario de la tabla:

Campo	Tipo	Restricción	Descripción
Dui	varchar(10)	Primary Key	Llave principal de Entrenador
Nombre	nvarchar(100)		Nombre completo del entrenador
Telefono	varchar(50)		Telefono de contacto del
Correo_electronico	varchar(150)		Correo electronico del entrenador
Salario	decimal(5,2)	Check (Salario ≥ 0)	Salario asignado al entrenador que
Seguro	varchar(100)		Tipo de seguro del entrenador

El tamaño estimado es de 0.009 MB para la tabla Entrenador

Tabla Reserva:

Columna	tipo de dato	Tamaño en bytes	Cálculo
Id_reserva	int	4	4
fecha_reserva	datetime2(0)	6	6
Id_socio	int	4	4
Id_clase	int	4	4
Overhead	N/A	7	fila variable

Tregistro=4 + 6 + 4 + 4 + 7 = 25 bytes

Ttabla=25×300,000 = 7,500,000 bytes

Convirtiendo a Megabytes: $7,500,000 \text{ bytes} \div 1024 = 7,324.22 \text{ KB}$

$7,324.22 \text{ KB} \div 1024 = 7.15 \text{ MB}$

Diccionario de tabla:

Campo	Tipo	Restricciones	Descripción
Id_reserva	int	Primary Key	Llave principal de la Reserva
fecha_reserva	datetime2(0)		Fecha-reserva yyyy-mm-dd
Id_socio	int	Foreign Key (Socio)	Llave foránea a Socio
Id_clase	int	Foreign Key (Clase)	Llave foránea a Clase

El tamaño estimado es de 7.15 MB para la tabla Reserva

Tabla TipoClase:

Columna	tipo de dato	Tamaño en bytes	Cálculo
Id_tipo_clase	varchar(200)	200	1 byte * 200
Nombre	varchar(150)	150	1 byte * 200
Overhead	N/A	7	fila variable

Tregistro=200+150+7 = 357 bytes

Ttabla=357×8=2,856 bytes

Convirtiendo a Megabytes: $2,856 \text{ bytes} \div 1024 = 2.79 \text{ KB}$

$2.79 \text{ KB} \div 1024 = 0.0027 \text{ MB}$

El tamaño estimado es de 0.0027 MB para la tabla TipoClase

Diccionario de la tabla:

Columna	Tipo	Restricción	Descripción
Id_tipo_clase	varchar(200)	Primary Key	Llave principal del tipoclase
Nombre	varchar(150)		Nombre del tipoclase

Tabla Socio:

Columna	Tipo de dato	Tamaño en byte	Cálculo
Id_socio	int	4	4
Nombre	nvarchar(150)	300	2 bytes * 150
Telefono	varchar(15)	15	1 byte * 15
Correo_electronico	varchar(150)	150	1 byte * 150
Fecha_registro	date	3	3
Id_plan	int	4	4
Overhead	N/A	7	fila variable

Tregistro= 4+300+15+150+3+4+7 = 483 bytes

Ttabla= 483×15,000 = 7,245,000 bytes

Convirtiendo a Megabytes: $7,245,000 \text{ bytes} \div 1024 = 7074.22 \text{ KB}$

$7074.22 \text{ KB} \div 1024 = 6.91 \text{ MB}$

Diccionario de tabla:

Columna	Tipo	Restricción	Descripción
Id_socio	int	Primary Key	Llave principal del Socio
Nombre	nvarchar(150)		Nombre completo del socio
Telefono	varchar(15)		Telefono del socio
Correo_electronico	varchar(150)		Correo electronico del socio
Fecha_registro	date		Fecha de registro del socio
Id_plan	int	Foreign Key (PlanGimnasio)	Llave foránea a PlanGimnasio

El tamaño estimado es de 6.91 MB para la tabla Socio

Tabla Factura:

Columna	Tipo de dato	Tamaño en bytes	Cálculo
Id_factura	int	4	4
Id_socio	int	4	4
Fecha	datetime	8	8
Monto	decimal	9	9
Metodo_pago	varchar(100)	100	1 byte * 100
Estado	varchar(100)	100	1 byte * 100
Overhead	N/A	7	fila variable

Tregistro= 4+4+8+9+100+100+7 = 232 bytes

Ttabla= 232×200,000 = 46,400,000 bytes

Convirtiendo a Megabytes: 46,400,000 bytes ÷ 1024 = 45,312.5 KB

45,312.5 KB ÷ 1024 = 44.27 MB

Diccionario de la tabla:

Campo	Tipo	Restricción	Descripción
Id_factura	int	Primary Key	Llave principal de la factura
Id_socio	int	Foreign Key (Socio)	Llave foránea a Socio
Fecha	datetime		Fecha-factura yyyy-mm-dd
Monto	decimal		Monto de la factura con 5
Metodo_pago	varchar(100)		Metodo de pago de cliente
Estado	varchar(100)		Estado de la factura

El tamaño estimado es de 44.27 MB para la tabla Factura

Tabla PlanGimnasio:

Columna	Tipo	Restricción	Descripción
Id_plan	int	Primary Key	Llave principal de PlanGimnasio
Nombre_plan	nvarchar(100)		Nombre del plan del Gimnasio
Descripcion	nvarchar(500)		Descripcion de los planes del Gimnasio
Costo	decimal(5,2)		Costo del plan del Gimnasio
Duracion_dias	int	Check (Duracion_dias ≥ 0)	Duracion de los dias del plan del

Tregistro= 4+200+1000+5+4+7 = 1220 bytes

Ttabla= 1220×6 = 7,320 bytes

Convirtiendo a Megabytes: 7,320 bytes ÷ 1024 = 7.15 KB

7.15 KB ÷ 1024 = 0.0069 MB

Diccionario de tabla:

Columna	Tipo	Restricción	Descripción
Id_plan	int	Primary Key	Llave principal de PlanGimnasio
Nombre_plan	nvarchar(100)		Nombre del plan del Gimnasio
Descripcion	nvarchar(500)		Descripcion de los planes del
Costo	decimal(5,2)		Costo del plan del Gimnasio
Duracion_dias	int	Check (Duracion_dias ≥	Duracion de los dias del plan del

El tamaño estimado es de 0.0069 MB para la tabla PlanGimnasio

Tamaño total:

Tabla	Peso
Clase	14.38 MB
Entrenador	0.009 MB
Reserva	7.15 MB
TipoClase	0.0027 MB
Socio	6.91 MB
Factura	44.27 MB
PlanGimnasio	0.0069 MB
Total	69.7786 MB

Migración e importación de datos

La base de datos fue poblada con datos generados sintéticamente a través de la librería faker de Python, esto permitió generar un dataset limpio, reproducible y controlado. Con el script de Python se generaron archivos CSV para las siete tablas de la base de datos.

La importación hacia SQL Server se realizó usando BULK INSERT dentro de procedimientos almacenados, se optó por esta opción por ser una opción robusta para cargas internas que permite manejar errores con TRY CATCH, controlar el uso de IDENTITY_INSERT y garantizar el orden

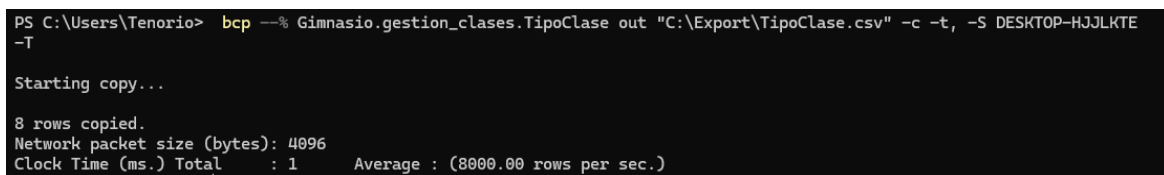
correcto entre las tablas con claves foráneas. Se creó un procedimiento por tabla y un procedimiento maestro que con una sola llamada carga todas las tablas desde archivos CSV.

Para la exportación, se utiliza otra herramienta fundamental: Los Bulk Copy Program (BCP), que permite exportar archivos desde fuera del servidor, permitiendo interoperabilidad con otros sistemas y crear respaldos sin depender de SQL Server Management Studio. Para exportar una tabla a CSV con autenticación de Windows utilizamos el siguiente comando modelo en la consola de Windows Powershell:

bcp --% NombreEsquema.NombreTabla out "C:\ruta\archivo.csv" -c -t, -S <Servidor> -d Base -T

Ya con los datos adecuados para nuestra base y servidor en específico:

bcp --% gimnasio.gestion_clases.TipoClase out "C:\exports\TipoClase.csv" -c -t, -S DESKTOP-HJJLKTE -d Gimnasio -T



```
PS C:\Users\Tenorio> bcp --% Gimnasio.gestion_clases.TipoClase out "C:\Export\TipoClase.csv" -c -t, -S DESKTOP-HJJLKTE -T
Starting copy...
8 rows copied.
Network packet size (bytes): 4096
Clock Time (ms.) Total : 1 Average : (8000.00 rows per sec.)
```

bcp --% Gimnasio.gestion_clases.TipoClase out "C:\Export\TipoClase.csv" -c -t, -S DESKTOP-HJJLKTE -T

bcp --% Gimnasio.gestion_clases.Entrenador out "C:\Export\Entrenador.csv" -c -t, -S DESKTOP-HJJLKTE -T

bcp --% Gimnasio.gestion_clases.Clase out "C:\Export\Clase.csv" -c -t, -S DESKTOP-HJJLKTE -T

bcp --% Gimnasio.gestion_clases.Reserva out "C:\Export\Reserva.csv" -c -t, -S DESKTOP-HJJLKTE -T

bcp --% Gimnasio.gestion_socios.Socio out "C:\Export\Socio.csv" -c -t, -S DESKTOP-HJJLKTE -T

bcp --% Gimnasio.gestion_socios.PlanGimnasio out "C:\Export\PlanGimnasio.csv" -c -t, -S DESKTOP-HJJLKTE -T

bcp --% Gimnasio.contabilidad.Factura out "C:\Export\Factura.csv" -c -t, -S DESKTOP-HJJLKTE -T

Si se desea importar nuevamente con Bulk Copy se utilizan los archivos csv generados previamente con el siguiente formato:

bcp --% Gimnasio.Esquema.Tabla in "C:\ruta\archivo.csv" -c -t, -S <SERVIDOR> -T

bcp --% Gimnasio.gestion_clases.TipoClase in "C:\Export\TipoClase.csv" -c -t, -S DESKTOP-HJJLKTE -T

SET IDENTITY_INSERT gestion_socios.PlanGimnasio ON;

bcp --% Gimnasio.gestion_socios.PlanGimnasio in "C:\Export\PlanGimnasio.csv" -c -t, -S DESKTOP-HJJLKTE -T

SET IDENTITY_INSERT gestion_socios.PlanGimnasio OFF;

SET IDENTITY_INSERT gestion_socios.Socio ON;

```
bcp --% Gimnasio.gestion_socios.Socio in "C:\Export\Socio.csv" -c -t, -S DESKTOP-HJJLKTE -T
```

```
SET IDENTITY_INSERT gestion_socios.Socio OFF;
```

```
bcp gestion_clases.TipoClase in "C:\Export\TipoClase.csv" -c -t, -S DEKTOP-HJJKTE -d Gimnasio -T
```

```
PS C:\Users\Tenorio> bcp --% Gimnasio.gestion_clases.Entrenador in "C:\Export\Entrenador.csv" -c -t, -S DESKTOP-HJJLKTE -T

Starting copy...

20 rows copied.
Network packet size (bytes): 4096
Clock Time (ms.) Total      : 63      Average : (317.46 rows per sec.)
PS C:\Users\Tenorio> |
```

El enfoque híbrido de Python, Bulk Insert y Bcp permite un pipeline completo y controlado de población, gestión y extracción de datos.

Evidencia del Dashboard en Power BI.

